Diseño e implementación de un taller-laboratorio práctico de nube privada y seguridad de la información en la nube sobre Kali Linux

[Nombre completo del autor 1] [Nombre completo del autor 2]

[Programa académico], [Facultad]

[Nombre de la universidad]

[Código y nombre de la asignatura]

[Nombre del docente]

10 de octubre de 2026

Guía del taller publicada en GitHub: https://github.com/[usuario]/taller-cloud-seguridad

Resumen

El presente documento describe el diseño, la implementación y la validación de un taller-laboratorio práctico de tres horas y cuarenta y cinco minutos orientado a la seguridad de la información en entornos de computación en la nube. El taller construye una nube privada sobre Kali Linux mediante contenedores Docker, compuesta por un servicio de almacenamiento de objetos compatible con Amazon S3 (MinIO) y un emulador de las interfaces de programación de Amazon Web Services (Moto). Sobre esa infraestructura se desarrollan cinco prácticas que reproducen, detectan y corrigen las amenazas más frecuentes identificadas por la Cloud Security Alliance: configuración errónea del almacenamiento, gestión deficiente de identidades y accesos, filtración de credenciales en repositorios de código, escalada de privilegios, uso de componentes sin parches e infraestructura como código insegura. Cada práctica se vincula con los controles de la norma ISO/IEC 27002:2022 y del catálogo NIST SP 800-53, y se acompaña de preguntas de análisis, evidencias esperadas y un solucionario. El material completo se publica como guía en un repositorio de GitHub, cuyos archivos se transcriben en los anexos. La validación realizada confirma el funcionamiento de los componentes emulados de AWS y la corrección sintáctica de los scripts y políticas, y deja documentadas las verificaciones pendientes sobre el equipo final.

Palabras clave: computación en la nube, nube privada, seguridad de la información, control de acceso, gestión de secretos, Kali Linux.

Abstract

This document presents the design, implementation and validation of a hands-on, half-day laboratory workshop on cloud information security. A private cloud is built on Kali Linux with Docker, combining S3-compatible object storage (MinIO) and an AWS API emulator (Moto). Five practices reproduce, detect and remediate the most frequent cloud threats: storage misconfiguration, weak identity and access management, leaked credentials, privilege escalation, unpatched components and insecure infrastructure as code. Each practice is mapped to ISO/IEC 27002:2022 and NIST SP 800-53 controls. The complete material is published as a GitHub repository and transcribed in the

annexes.

Keywords: cloud computing, private cloud, information security, access control, secrets management, Kali Linux.

Tabla de contenido

Resumen **2**

Abstract **2**

Introducción **7**

Objetivos **8**
Objetivo general 8
Objetivos específicos 8

Marco teórico **8**
Computación en la nube 8
Modelo de responsabilidad compartida 9
Seguridad de la información y marco normativo 9
Amenazas principales en la nube 10
Control de acceso y mínimo privilegio 11
Gestión de secretos 11
Seguridad de contenedores e infraestructura como código 11
Marco legal colombiano 12

Metodología del taller **12**
Enfoque pedagógico 12
Población y requisitos 13
Arquitectura de la nube privada 13
Herramientas 15
Agenda 16
Consideraciones éticas 16

Desarrollo de las prácticas **17**
Laboratorio 1: despliegue de la nube privada y superficie de ataque 17
Laboratorio 2: almacenamiento mal configurado 18
Laboratorio 3: identidad y mínimo privilegio 20
Laboratorio 4: secretos filtrados y auditoría de identidades 22
Laboratorio 5: auditoría y aseguramiento 23

Evaluación del aprendizaje **25**

Matriz de trazabilidad de controles **26**

Validación técnica **26**

Conclusiones **27**

Referencias **29**

Anexo A. Estructura del repositorio y publicación en GitHub **32**

Anexo B. Preparación del entorno en Kali Linux **34**

Anexo C. Definición de la nube privada **38**

Anexo D. Scripts de operación del laboratorio **40**

Anexo E. Políticas de control de acceso **43**

Anexo F. Scripts del escenario de secretos y auditoría de identidades **46**

Anexo G. Infraestructura como código con errores deliberados **51**

Anexo H. Solucionario de las preguntas de análisis **54**

Anexo I. Lista de verificación de seguridad en la nube **57**

Anexo J. Guía del facilitador y reto final **59**

Anexo K. Evidencias de la validación técnica **61**

Índice de tablas

1 Principales amenazas de la computación en la nube (2024) y su cobertura en el taller 10
2 Requisitos técnicos del equipo del participante 13
3 Componentes de la nube privada y su equivalente en la nube pública 15
4 Herramientas del taller 15
5 Agenda del taller 16
6 Resultados esperados de las pruebas de mínimo privilegio 21
7 Rúbrica de evaluación 25
8 Matriz de trazabilidad entre laboratorios, amenazas y controles 26
9 Problemas frecuentes durante la preparación y su solución 35
10 Hallazgos y correcciones esperadas en la infraestructura como código 56
11 Mensajes clave y errores frecuentes por bloque 59

Índice de figuras

1 Ciclo de aprendizaje aplicado en cada laboratorio 12
2 Arquitectura de la nube privada del taller 14

Introducción

La adopción de servicios en la nube ha desplazado el perímetro de seguridad de las organizaciones. La protección ya no depende solo de un cortafuegos que separe la red interna de Internet. Ahora depende de configuraciones, identidades y credenciales que se gestionan mediante interfaces de programación accesibles desde cualquier lugar. En su informe de amenazas de 2024, la Cloud Security Alliance ubicó la configuración errónea y el control inadecuado de cambios en el primer lugar, seguidos de la gestión de identidades y accesos y de las interfaces inseguras (Cloud Security Alliance, 2024). Ninguna de estas amenazas exige vulnerar al proveedor: todas se originan en decisiones del cliente.

Esta realidad plantea un reto formativo. Los estudiantes de seguridad de la información suelen conocer estos riesgos en teoría, pero rara vez los han provocado, observado y corregido en un entorno controlado. Además, practicar sobre una cuenta real de un proveedor público implica costos, requiere tarjeta de crédito y conlleva el riesgo de exponer recursos por error.

Para responder a ese reto, el presente documento expone el diseño de un taller-laboratorio que construye una nube privada completa dentro de una máquina con Kali Linux, sin costo y sin conexión a sistemas de terceros. Sobre ella se desarrollan cinco prácticas de seguridad de la información con un enfoque ofensivo-defensivo. Primero se provoca el error, después se explota como lo haría un atacante, a continuación se detecta como lo haría un equipo de monitoreo y por último se corrige aplicando un control normalizado.

El taller se publica como guía en un repositorio de GitHub (https://github.com/[usuario]/taller-cloud-seguridad). Ese repositorio contiene los archivos de despliegue, los scripts y las instrucciones de cada laboratorio. Este documento constituye su soporte académico: justifica el diseño, lo relaciona con el marco normativo y transcribe en los anexos todos los archivos del repositorio, de modo que el taller pueda reproducirse a partir del documento.

El documento se organiza de la siguiente manera. Primero se plantean los objetivos y el marco teórico. A continuación se describe la metodología del taller y se desarrollan las cinco prácticas. Después se presentan la evaluación del aprendizaje, la matriz de trazabilidad con los controles de

seguridad y la validación técnica realizada. Por último se exponen las conclusiones. Los anexos A a K contienen la estructura del repositorio, los procedimientos de instalación, el código fuente, el solucionario, la lista de verificación, la guía del facilitador y las evidencias de validación.

Objetivos

Objetivo general

Diseñar e implementar un taller-laboratorio práctico de medio día que permita a los participantes desplegar una nube privada sobre Kali Linux y aplicar sobre ella controles de seguridad de la información frente a las amenazas más frecuentes de los entornos en la nube.

Objetivos específicos

1. Desplegar una nube privada con almacenamiento de objetos y servicios emulados de AWS, e identificar su superficie de ataque (Laboratorio 1).
2. Reproducir la exposición de datos por almacenamiento público, enumerarla sin credenciales y corregirla con mecanismos de acceso temporal y versionado (Laboratorio 2).
3. Diseñar y verificar políticas de control de acceso basadas en el principio de mínimo privilegio, incluido el uso del rechazo explícito (Laboratorio 3).
4. Detectar credenciales filtradas en un repositorio de código, gestionar el incidente y auditar permisos que permiten escalar privilegios (Laboratorio 4).
5. Auditar imágenes de contenedores e infraestructura como código, y analizar los registros de actividad de la nube (Laboratorio 5).

Marco teórico

Computación en la nube

El Instituto Nacional de Estándares y Tecnología de los Estados Unidos define la computación en la nube como un modelo que habilita el acceso ubicuo, conveniente y bajo demanda a un conjunto compartido de recursos configurables, que pueden aprovisionarse y liberarse con un

esfuerzo de gestión mínimo (Mell & Grance, 2011). La definición reconoce cinco características esenciales: autoservicio bajo demanda, amplio acceso a la red, agrupación de recursos, elasticidad rápida y servicio medido. También distingue tres modelos de servicio (software, plataforma e infraestructura como servicio) y cuatro modelos de despliegue: nube privada, comunitaria, pública e híbrida.

La nube privada es aquella cuya infraestructura se aprovisiona para el uso exclusivo de una sola organización. Puede ser operada por la propia organización o por un tercero, y puede estar dentro o fuera de sus instalaciones (Mell & Grance, 2011). El taller adopta este modelo en miniatura: la infraestructura completa se ejecuta en el equipo del participante, que actúa simultáneamente como proveedor y como cliente.

Modelo de responsabilidad compartida

Los proveedores públicos dividen las obligaciones de seguridad en dos ámbitos. El proveedor protege la infraestructura que soporta los servicios: instalaciones físicas, hardware, red y virtualización. Esta es la seguridad *de* la nube. El cliente protege lo que construye y almacena en ella: datos, aplicaciones, identidades y configuraciones. Esta es la seguridad *en* la nube (Amazon Web Services, 2026c). Las amenazas que ocupan los primeros lugares del informe de la Cloud Security Alliance pertenecen al ámbito del cliente (Cloud Security Alliance, 2024). Por eso el taller concentra en él todas sus prácticas.

Seguridad de la información y marco normativo

La seguridad de la información persigue la preservación de la confidencialidad, la integridad y la disponibilidad de la información. La norma ISO/IEC 27001:2022 establece los requisitos de un sistema de gestión de seguridad de la información (International Organization for Standardization & International Electrotechnical Commission [ISO/IEC], 2022a). Su complemento, ISO/IEC 27002:2022, describe 93 controles agrupados en cuatro temas: organizacionales (37), de personas (8), físicos (14) y tecnológicos (34) (ISO/IEC, 2022b). La edición de 2022 incorporó el control 5.23, dedicado específicamente a la seguridad de la información en el uso de servicios en la nube. La nor-

ma ISO/IEC 27017:2015 amplía la guía de implementación para proveedores y clientes de servicios en la nube (ISO/IEC, 2015).

De forma complementaria, la publicación NIST SP 800-53 Rev. 5 ofrece un catálogo de controles de seguridad y privacidad para sistemas de información (Joint Task Force, 2020). Entre ellos se encuentra la familia de control de acceso (AC), a la que pertenece el control AC-6 de mínimo privilegio. Las prácticas del taller se relacionan con ambos marcos, como se detalla en el apartado «Matriz de trazabilidad de controles».

Amenazas principales en la nube

La Tabla 1 presenta las once amenazas principales identificadas por la Cloud Security Alliance en 2024 y señala el laboratorio del taller que aborda cada una. La matriz ATT&CK de MITRE para entornos en la nube complementa esta visión con las tácticas y técnicas observadas en ataques reales contra infraestructura como servicio, software como servicio y proveedores de identidad (MITRE Corporation, 2026).

Tabla 1

Principales amenazas de la computación en la nube (2024) y su cobertura en el taller

N.º	Amenaza	Laboratorio
1	Configuración errónea y control inadecuado de cambios	1, 2, 5
2	Gestión de identidades y accesos (IAM)	3, 4
3	Interfaces y API inseguras	1, 2
4	Selección o implementación inadecuada de la estrategia de seguridad	Cierre
5	Recursos inseguros de terceros	5
6	Desarrollo de software inseguro	4, 5
7	Divulgación accidental de datos en la nube	2, 4
8	Vulnerabilidades del sistema	5
9	Visibilidad u observabilidad limitada	3, 5
10	Compartición de recursos sin autenticación	2
11	Amenazas persistentes avanzadas	—

Nota. Adaptado de Cloud Security Alliance (2024). La asignación de laboratorios es elaboración propia.

Control de acceso y mínimo privilegio

El principio de mínimo privilegio establece que cada usuario o proceso debe disponer únicamente de los permisos necesarios para cumplir su función (Joint Task Force, 2020). En la nube, este principio se materializa mediante políticas que combinan una acción, un recurso y un efecto (permitir o denegar). La guía NIST SP 800-210 analiza los retos del control de acceso en los distintos modelos de servicio (Hu et al., 2020).

La lógica de evaluación de políticas de AWS sigue tres reglas. Toda solicitud se deniega por defecto. Una política que permite la acción la habilita. Un rechazo explícito prevalece sobre cualquier permiso, sin importar qué política lo conceda (Amazon Web Services, 2026b). El servicio MinIO implementa el mismo lenguaje de políticas, lo que permite comprobar esta lógica en el Laboratorio 3.

Gestión de secretos

Las claves de acceso, contraseñas y certificados son secretos que dan acceso a los recursos de la nube. La guía de OWASP recomienda centralizarlos en un gestor especializado y restringir su lectura con controles de acceso granulares. También recomienda rotarlos de forma automática, auditar cada uso y detectar su exposición con herramientas de escaneo. Ante una filtración, la respuesta comienza por revocar y rotar la credencial (OWASP Foundation, 2026). El Laboratorio 4 reproduce este ciclo completo.

Seguridad de contenedores e infraestructura como código

La guía NIST SP 800-190 identifica como riesgos principales de los contenedores las vulnerabilidades en las imágenes, las configuraciones inseguras y los secretos embebidos (Souppaya et al., 2017). A estos riesgos se suma la infraestructura como código. Una plantilla con errores replica la misma configuración insegura en cada despliegue. Un ejemplo relevante es el servicio de metadatos de instancias de AWS: su versión 2 exige un token de sesión que dificulta el robo de credenciales mediante falsificación de solicitudes del lado del servidor (SSRF) (Amazon Web

Services, 2026d). El Laboratorio 5 aplica escaneo automatizado a ambos tipos de artefacto.

Marco legal colombiano

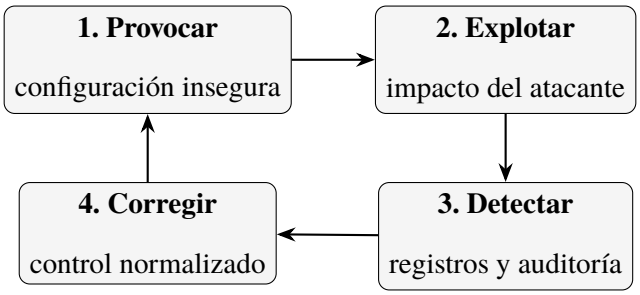
Dos normas colombianas enmarcan las prácticas. La Ley 1273 de 2009 tipifica, entre otros, los delitos de acceso abusivo a un sistema informático y de violación de datos personales (Congreso de la República de Colombia, 2009). Por ello todas las técnicas ofensivas del taller se aplican exclusivamente sobre la infraestructura propia del participante. La Ley Estatutaria 1581 de 2012 regula el tratamiento de datos personales (Congreso de la República de Colombia, 2012). Los datos de nómina y de clientes que se exponen en los Laboratorios 2 y 4 son ficticios, pero ilustran la clase de información cuya divulgación genera responsabilidad legal para una organización.

Metodología del taller

Enfoque pedagógico

El taller sigue un enfoque de aprendizaje basado en la práctica, estructurado en un ciclo de cuatro fases que se repite en cada laboratorio (Figura 1). En la fase de *provocar*, el participante reproduce una configuración insegura habitual. En la de *explotar*, actúa como atacante y comprueba el impacto. En la de *detectar*, observa la actividad como lo haría un equipo de monitoreo. En la de *corregir*, aplica el control de seguridad y verifica su eficacia.

Figura 1
Ciclo de aprendizaje aplicado en cada laboratorio



Nota. Elaboración propia.

Población y requisitos

El taller se dirige a estudiantes y profesionales de nivel intermedio, con conocimientos básicos de Linux, redes y línea de comandos, sin experiencia previa en la nube. La Tabla 2 resume los requisitos técnicos del equipo de cada participante. El procedimiento completo de instalación se presenta en el Anexo B.

Tabla 2
Requisitos técnicos del equipo del participante

Recurso	Mínimo	Recomendado
Sistema operativo	Kali Linux 2025.x (física, máquina virtual o WSL2)	Kali Linux actualizada
Memoria RAM	4 GB	8 GB
Disco libre	5 GB	10 GB
Procesador	2 núcleos	4 núcleos
Conexión a Internet	Solo durante la instalación previa	—

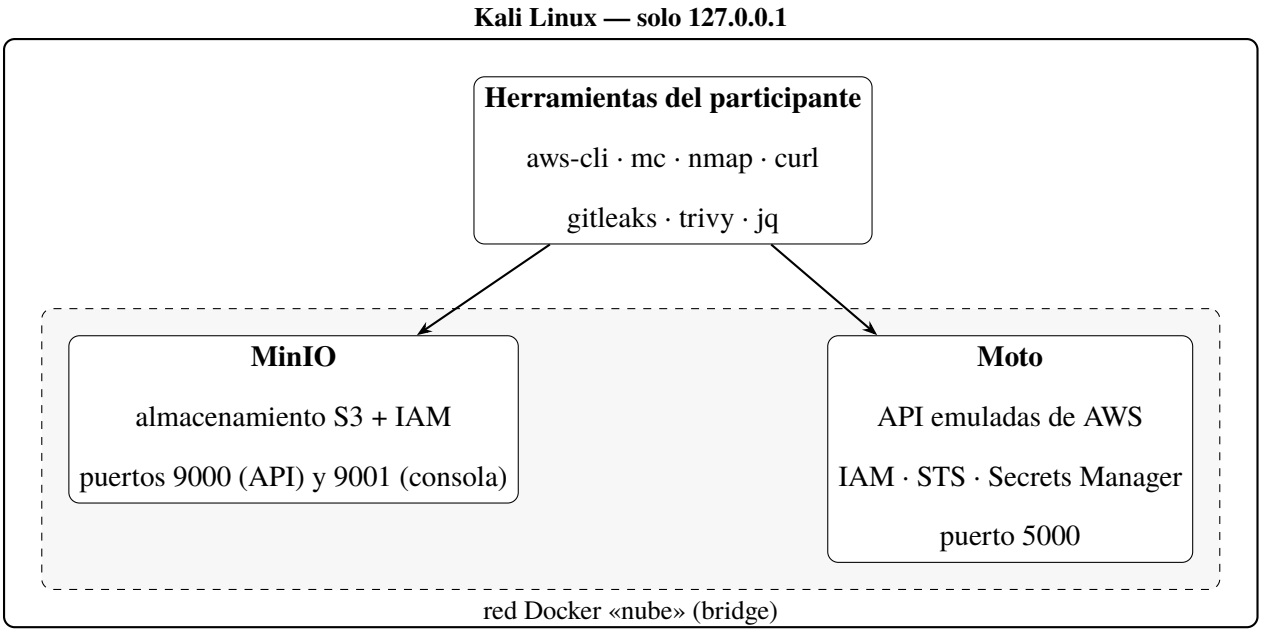
Nota. Elaboración propia.

Arquitectura de la nube privada

La nube privada se compone de dos servicios en contenedores conectados por una red interna de Docker (Figura 2). La decisión de diseño principal es publicar todos los puertos únicamente en la interfaz local (`127.0.0.1`). Así la nube no queda expuesta a la red del aula. La definición completa se encuentra en el archivo `docker-compose.yml`, transcrito en el Anexo C.

Figura 2

Arquitectura de la nube privada del taller



Nota. Elaboración propia.

La Tabla 3 relaciona cada componente con su equivalente en un proveedor público. La selección de los componentes respondió a dos cambios recientes en el ecosistema de código abierto. Primero, el emulador LocalStack exige desde marzo de 2026 una cuenta y un token de autenticación para iniciar su contenedor, incluso en su plan gratuito (LocalStack, 2026). Por esa razón se adoptó Moto, de licencia Apache 2.0 y sin registro (Moto Developers, 2026). Segundo, la edición comunitaria de MinIO pasó a modo de mantenimiento y dejó de publicar imágenes de contenedor (MinIO, Inc., 2025). Se utiliza la última imagen publicada, fijada por versión. Esta circunstancia se aprovecha en el Laboratorio 5 como caso de estudio sobre los riesgos de la cadena de suministro de software.

Tabla 3
Componentes de la nube privada y su equivalente en la nube pública

Componente	Función en el taller	Equivalente en AWS
MinIO	Almacenamiento de objetos con usuarios y políticas que se aplican efectivamente	Amazon S3 y AWS IAM
Moto	Emulación de las API de identidad, credenciales temporales y gestión de secretos	Cuenta de AWS de pruebas
Docker	Ejecución y aislamiento de los servicios	Capa de cómputo del proveedor
Red «nube»	Comunicación interna entre servicios	Nube privada virtual (VPC)

Nota. Elaboración propia.

Herramientas

La Tabla 4 presenta las herramientas que utiliza el participante. Todas se instalan desde los repositorios de Kali Linux o desde su sitio oficial mediante el script del Anexo B. Para Docker se sigue la recomendación de la documentación de Kali, que indica instalar el paquete `docker.io` (OffSec Services Limited, 2026).

Tabla 4
Herramientas del taller

Herramienta	Uso en el taller	Laboratorio
Docker y Docker Compose	Despliegue de la nube privada	1–5
nmap, ss, curl	Reconocimiento y peticiones anónimas a la API	1, 2
mc (cliente de MinIO)	Administración de almacenamiento, usuarios y políticas	2, 3, 4, 5
AWS CLI	Consulta de las API emuladas de AWS	4
jq	Análisis de documentos JSON	4, 5
gitleaks	Detección de secretos en repositorios Git (Gitleaks Contributors, 2026)	4
Trivy	Escaneo de imágenes e infraestructura como código (Aqua Security, 2026)	5

Nota. Elaboración propia.

Agenda

El taller tiene una duración total de tres horas y cuarenta y cinco minutos, distribuida según la Tabla 5. La guía del facilitador, con los mensajes clave y los errores típicos de cada bloque, se incluye en el Anexo J.

Tabla 5
Agenda del taller

Horario	Bloque	Temas de seguridad
0:00–0:20	Introducción	Modelos de servicio, responsabilidad compartida y amenazas principales
0:20–0:50	Laboratorio 1	Superficie de ataque, exposición de puertos y reconocimiento
0:50–1:35	Laboratorio 2	Almacenamiento público, enumeración anónima, URL prefirmadas y versionado
1:35–1:45	Pausa	—
1:45–2:25	Laboratorio 3	Cuenta raíz, mínimo privilegio, rechazo explícito y revocación
2:25–3:00	Laboratorio 4	Secretos filtrados, respuesta a incidentes y escalada de privilegios
3:00–3:30	Laboratorio 5	Imágenes vulnerables, infraestructura como código y registros
3:30–3:45	Cierre	Lista de verificación, reto final y preguntas

Nota. Elaboración propia.

Consideraciones éticas

Las técnicas ofensivas del taller se ejecutan exclusivamente contra la nube privada desplegada en el equipo de cada participante. Antes del primer laboratorio, el facilitador advierte que aplicarlas sobre sistemas ajenos sin autorización escrita constituye un delito según la Ley 1273 de 2009 (Congreso de la República de Colombia, 2009). Todos los datos personales utilizados son ficticios. Las credenciales que se generan solo funcionan dentro del laboratorio. Las claves con formato de AWS del Laboratorio 4 se crean de forma aleatoria y no corresponden a ninguna cuenta real.

Desarrollo de las prácticas

Antes de cada laboratorio, el participante carga en su terminal las variables y los alias del entorno con el comando `source scripts/entorno.sh`. El alias `nube` del cliente `mc` apunta a MinIO con la cuenta raíz. El alias `awsm` ejecuta la AWS CLI contra Moto. Las instrucciones detalladas de cada laboratorio se publican en el repositorio, en las carpetas `labs/01-nube-privada` a `labs/05-auditoria` (Anexo A). Las respuestas a las preguntas de análisis se encuentran en el Anexo H.

Laboratorio 1: despliegue de la nube privada y superficie de ataque

Objetivo y duración

Levantar la nube privada, reconocer sus componentes y analizarla desde la perspectiva de un atacante. Duración: 30 minutos.

Amenaza y controles asociados

Configuración errónea e interfaces expuestas (Cloud Security Alliance, 2024). Controles ISO/IEC 27002:2022: 8.20 (seguridad de redes) y 8.9 (gestión de la configuración) (ISO/IEC, 2022b).

Procedimiento

1. Se despliegan los servicios y se carga el entorno con los scripts del Anexo D:

```
./scripts/lab.sh up
source scripts/entorno.sh
mc admin info nube
awsm sts get-caller-identity
```

2. Se inspecciona la red interna (`docker network inspect nube-taller_nube`) y se identifica la dirección de cada contenedor.

3. Se realiza el reconocimiento de los servicios publicados:

```
sudo ss -tlnp | grep -E ':(9000|9001|5000)'
nmap -sV -p 9000,9001,5000 127.0.0.1
curl -i http://localhost:9000/
```

4. Se simula el error de exposición. En `docker-compose.yml` se reemplaza la publicación `"127.0.0.1:9001:9001"` por `"9001:9001"`, se reinicia el servicio y se escanea la dirección IP de red del equipo. Se observa que el puerto pasa a escuchar en todas las interfaces.
5. Se revierte el cambio y se verifica que el puerto vuelve a estar limitado a la interfaz local.

Evidencia esperada

El escaneo inicial muestra los tres puertos abiertos solo en `127.0.0.1`. La cabecera `Server: MinIO` identifica el software. Tras el cambio del paso 4, la consola de MinIO es alcanzable desde la red local. Un hallazgo relevante es que Docker inserta sus propias reglas de filtrado de paquetes. Según su documentación, los puertos publicados se enrutan antes de que se apliquen las reglas de `ufw`, de modo que un cortafuegos activo en el equipo no impide la exposición (Docker Inc., 2026).

Preguntas de análisis

1. ¿Qué dirección interna tiene cada contenedor y cómo se comunican entre sí?
2. ¿Qué información aporta la cabecera del servidor a un atacante?
3. ¿Qué controles de un proveedor público evitarían la exposición simulada?

Laboratorio 2: almacenamiento mal configurado

Objetivo y duración

Reproducir la exposición de datos por un contenedor de almacenamiento público, enumerarlo sin credenciales y corregirlo. Duración: 45 minutos.

Amenaza y controles asociados

Divulgación accidental de datos y compartición de recursos sin autenticación (Cloud Security Alliance, 2024). Controles ISO/IEC 27002:2022: 5.15 (control de acceso), 8.3 (restricción del acceso a la información), 8.12 (prevención de fuga de datos) y 8.13 (copias de respaldo) (ISO/IEC, 2022b).

Procedimiento

1. *Provocar.* El administrador crea los contenedores `marketing` y `rrhh`, carga un borrador de campaña y un archivo de nómina ficticio, y aplica por error acceso anónimo público a ambos:

```
mc mb nube/marketing nube/rrhh
mc cp nomina.csv nube/rrhh/
mc anonymous set public nube/rrhh
```

2. *Explotar.* Sin credenciales, el participante prueba nombres de contenedores habituales y compara los códigos HTTP de respuesta. Después lista el contenido del contenedor abierto y descarga el archivo:

```
for b in backup datos rrhh nomina marketing finanzas; do
  echo "$b -> $(curl -s -o /dev/null -w '%{http_code}' http://localhost:9000/$b/)"
done
curl -s http://localhost:9000/rrhh/ | grep -oP '(?<=<Key>)[^<]+'
curl -s http://localhost:9000/rrhh/nomina.csv
```

   Además, se comprueba si es posible escribir de forma anónima con una petición `PUT`.

3. *Detectar.* En otra terminal se ejecuta `mc admin trace nube` y se repiten las peticiones anónimas para observar qué registra el servidor.

4. *Corregir.* Se retira el acceso anónimo (`mc anonymous set none`). Para compartir un único archivo con un tercero se genera una URL prefirmada con caducidad (`mc share download --expire 2m`). Por último se activa el versionado (`mc version enable nube/rrhh`) y se comprueba que una sobrescritura maliciosa no destruye la versión original.

Evidencia esperada

Antes de la corrección, el contenedor `rrhh` responde con código 200 y entrega la nómina a cualquier cliente. Después, la misma petición responde con código 403. La URL prefirmada funciona durante dos minutos y luego se rechaza. En Amazon S3, el bloqueo de acceso público aplicado a nivel de cuenta impide este error de raíz, porque prevalece sobre las políticas y listas de control de acceso que intenten conceder acceso público (Amazon Web Services, 2026a).

Preguntas de análisis

1. ¿Qué diferencia existe entre las políticas anónimas `download`, `upload` y `public`?
2. ¿Qué datos de la petición anónima registra el servidor?
3. ¿Por qué no es posible modificar la fecha de caducidad de una URL prefirmada?

Laboratorio 3: identidad y mínimo privilegio

Objetivo y duración

Sustituir el uso de la cuenta raíz por identidades con permisos mínimos y verificar la lógica de evaluación de políticas. Duración: 40 minutos.

Amenaza y controles asociados

Gestión de identidades y accesos (Cloud Security Alliance, 2024). Controles ISO/IEC 27002:2022: 5.15 (control de acceso), 5.18 (derechos de acceso) y 8.2 (derechos de acceso privilegiado) (ISO/IEC, 2022b). Control NIST SP 800-53: AC-6 (mínimo privilegio) (Joint Task Force, 2020).

Procedimiento

1. Se analiza la cuenta raíz y se revisan las políticas predefinidas de MinIO (`mc admin policy list nube`).
2. Se reproduce el atajo peligroso. Se crea el usuario `practicante` con la política `demasiado-permisiva`, que concede todas las acciones sobre todos los recursos, y se comprueba que puede leer la nómina.
3. Se aplica el mínimo privilegio. Se crea el usuario `analista` con la política `analista-lectura-informes`, que solo permite listar y leer el contenedor `informes`:

```
mc admin policy create nube analista-informes politicas/analista-lectura-informes.
    json
mc admin user add nube analista 'Analista-2026!'
mc admin policy attach nube analista-informes --user analista
mc alias set analista http://localhost:9000 analista 'Analista-2026!'
```

4. Se verifica la precedencia del rechazo explícito. Se crea el usuario `robot-backup` con la política `backup-sin-borrado` y, de forma deliberada, también con la política predefinida `readwrite`. Después se intenta borrar una copia de respaldo.

5. Se simula la respuesta a una filtración: se deshabilita al practicante (`mc admin user disable`), se retiran las políticas excesivas y se comprueba que su acceso deja de funcionar.

Las tres políticas utilizadas se transcriben en el Anexo E.

Evidencia esperada

La Tabla 6 resume los resultados que debe obtener el participante con la identidad `analista`. El usuario `robot-backup` puede escribir copias, pero no borrarlas, aunque tenga asignada la política `readwrite`. Este resultado confirma que el rechazo explícito prevalece sobre cualquier permiso (Amazon Web Services, 2026b).

Tabla 6
Resultados esperados de las pruebas de mínimo privilegio

Prueba	**Comando**	**Resultado**
Listar el contenedor de informes	`mc ls analista/informes`	Permitido
Leer un informe	`mc cat analista/informes/q3.txt`	Permitido
Modificar un informe	`mc pipe analista/informes/q3.txt`	Denegado
Leer la nómina	`mc cat analista/rrhh/nomina.csv`	Denegado
Listar todos los contenedores	`mc ls analista/`	Denegado

Nota. Elaboración propia.

Preguntas de análisis

1. ¿Qué puede hacer un atacante que obtiene las credenciales de la cuenta raíz?

2. ¿Por qué la acción `s3:ListBucket` se aplica sobre el contenedor y `s3:GetObject` sobre sus objetos?

3. ¿Cuál es la regla de evaluación cuando coinciden un permiso y un rechazo explícito?

Laboratorio 4: secretos filtrados y auditoría de identidades

Objetivo y duración

Detectar credenciales filtradas en el historial de un repositorio, medir su impacto, gestionar el incidente y auditar permisos que permiten escalar privilegios. Duración: 35 minutos.

Amenaza y controles asociados

Desarrollo de software inseguro y gestión de identidades (Cloud Security Alliance, 2024). Controles ISO/IEC 27002:2022: 5.17 (información de autenticación), 5.26 (respuesta a incidentes), 8.4 (acceso al código fuente) y 8.28 (codificación segura) (ISO/IEC, 2022b).

Preparación del escenario

El script `preparar-escenario.sh` crea en MinIO una cuenta de servicio con acceso a datos ficticios de clientes. Después genera un repositorio Git en el que un desarrollador escribe esas credenciales en el código y, en un segundo commit, las retira. El script `poblar-iam-aws.sh` crea en Moto usuarios y políticas, algunas de ellas peligrosas, junto con un secreto almacenado correctamente. Ambos scripts se transcriben en el Anexo F.

Procedimiento

1. Se comprueba que la versión actual del código no contiene credenciales y se buscan en el historial, primero de forma manual y luego con gitleaks:

```
git log -p | grep -nE 'ACCESS_KEY|SECRET_KEY'
gitleaks detect --source . -v
```

2. Se mide el impacto: con las credenciales encontradas se configura un alias del cliente y se leen los datos de clientes.

3. Se gestiona el incidente en el orden correcto. Primero se revoca la credencial (`mc admin user disable`). Después se verifica que dejó de funcionar y se revisa la actividad registrada. Por último se planifican la rotación, el traslado a un gestor de secretos y la limpieza del historial.

4. Se consulta el secreto almacenado en el gestor de secretos emulado (`awsm secretsmanager`

`get-secret-value`).

5. Se ejecuta el auditor de identidades `auditar-iam.sh`, que revisa las políticas gestionadas por el cliente y señala comodines totales, acciones que permiten escalar privilegios y usuarios administradores con claves de larga duración.

Evidencia esperada

gitleaks identifica la clave con formato `AKIA...` en el primer commit, aunque el último commit ya no la contenga. Tras la revocación, el alias con las credenciales filtradas recibe un error de autenticación. El auditor reporta tres hallazgos. La política `AdminTodo` concede todas las acciones sobre todos los recursos. La política `DevGestionaIAM` permite a su titular asignarse cualquier otra política. El usuario `bot-ci` combina privilegios de administrador con una clave de acceso permanente. La salida real de esta ejecución se documenta en el Anexo K. Este resultado coincide con la recomendación de OWASP de revocar y rotar como primera respuesta ante una exposición (OWASP Foundation, 2026).

Preguntas de análisis

1. ¿Por qué el segundo commit no resuelve la filtración?
2. ¿Qué permiso exacto necesita una aplicación para leer su secreto del gestor y sobre qué recurso?
3. ¿Cómo podría el usuario `diego-dev` convertirse en administrador de la cuenta?

Laboratorio 5: auditoría y aseguramiento

Objetivo y duración

Auditar la nube como lo haría un equipo de seguridad: imágenes de contenedor, infraestructura como código y registros de actividad. Duración: 30 minutos.

Amenaza y controles asociados

Vulnerabilidades del sistema, recursos inseguros de terceros y visibilidad limitada (Cloud Security Alliance, 2024). Controles ISO/IEC 27002:2022: 8.8 (gestión de vulnerabilidades técnicas), 8.9 (gestión de la configuración), 8.15 (registro) y 8.16 (seguimiento de actividades) (ISO/IEC, 2022b).

Procedimiento

1. Se escanea la imagen de MinIO utilizada por la nube y se cuantifican las vulnerabilidades por severidad:

```
trivy image --severity HIGH,CRITICAL minio/minio:RELEASE.2025-09-07T16-13-09Z
```

2. Se analiza la plantilla Terraform del Anexo G, que contiene siete errores deliberados, con `trivy config terraform/`. Se completa la tabla de hallazgos y se elabora una versión corregida hasta eliminar los hallazgos injustificados.

3. Se registra la actividad de la API con `mc admin trace -v nube`, se generan accesos permitidos, denegados y anónimos, y se identifican los campos útiles para crear una alerta.

Evidencia esperada

Trivy reporta vulnerabilidades de severidad alta o crítica en la imagen. Este resultado es coherente con la ausencia de nuevas publicaciones de la edición comunitaria (MinIO, Inc., 2025) y ejemplifica el riesgo que la guía NIST SP 800-190 asocia a las imágenes desactualizadas (Souppaya et al., 2017). Sobre la plantilla, Trivy señala varios problemas: el bloqueo de acceso público desactivado, el acceso SSH abierto a Internet, el servicio de metadatos en su versión 1, el disco sin cifrar y la política con comodín total. Todos ellos tienen una corrección documentada en el Anexo H. La traza permite identificar la identidad, la operación, el recurso, el código de respuesta y el origen de cada solicitud.

Preguntas de análisis

1. ¿Qué opciones tiene una organización que depende de un componente que dejó de recibir actualizaciones?
2. ¿Qué campos de la traza sustentarían una alerta de acceso no autorizado a datos de recursos humanos, y qué servicios cumplen esa función en AWS?

Evaluación del aprendizaje

El desempeño de cada participante se evalúa con la rúbrica de la Tabla 7, a partir de las evidencias de cada laboratorio y de las respuestas a las preguntas de análisis. Como actividad de cierre, el facilitador propone un reto en el que se introducen tres configuraciones inseguras ocultas: un contenedor con acceso anónimo, un usuario con privilegios administrativos innecesarios y una cuenta huérfana de un exempleado. El participante debe encontrarlas y corregirlas con las herramientas vistas. El procedimiento del reto se describe en el Anexo J. La lista de verificación del Anexo I sirve como instrumento de autoevaluación y de transferencia a la organización del participante.

Tabla 7
Rúbrica de evaluación

Criterio	Destacado (5)	Suficiente (3)	Insuficiente (1)	Peso
Ejecución técnica	Completa los cinco laboratorios sin asistencia	Completa al menos tres laboratorios	Completa menos de tres	30 %
Análisis	Responde las preguntas relacionando causa, impacto y control	Responde con explicaciones parciales	Responde sin justificar	30 %
Corrección	Aplica y verifica todos los controles	Aplica los controles sin verificarlos	No aplica los controles	25 %
Reto final	Encuentra y corrige los tres errores	Encuentra dos errores	Encuentra uno o ninguno	15 %

Nota. Elaboración propia.

Matriz de trazabilidad de controles

La Tabla 8 relaciona cada laboratorio con la amenaza que aborda, los controles de ISO/IEC 27002:2022 (ISO/IEC, 2022b) y NIST SP 800-53 (Joint Task Force, 2020) que aplica, y la evidencia que demuestra su eficacia. La matriz permite justificar el taller dentro de un sistema de gestión de seguridad de la información conforme a ISO/IEC 27001:2022 (ISO/IEC, 2022a). También guía la selección de prácticas cuando el tiempo disponible es menor.

Tabla 8

Matriz de trazabilidad entre laboratorios, amenazas y controles

Lab.	Amenaza (CSA 2024)	ISO/IEC 27002:2022	NIST 800-53	Evidencia de eficacia
1	Configuración errónea; API inseguras	8.9 Gestión de la configuración; 8.20 Seguridad de redes	CM-7, SC-7	Puertos limitados a 127.0.0.1 según nmap
2	Divulgación accidental; recursos sin autenticación	5.15 Control de acceso; 8.12 Prevención de fuga; 8.13 Respaldo	AC-3, CP-9	Código 403 al acceso anónimo; versión original recuperada
3	Gestión de identidades y accesos	5.18 Derechos de acceso; 8.2 Acceso privilegiado	AC-2, AC-6	Tabla 6; borrado denegado al robot
4	Software inseguro; identidades	5.17 Información de autenticación; 5.26 Respuesta a incidentes; 8.28 Codificación segura	IA-5, IR-4	Secreto detectado y revocado; tres hallazgos del auditor
5	Vulnerabilidades; terceros; visibilidad	8.8 Vulnerabilidades técnicas; 8.15 Registro; 8.16 Seguimiento	RA-5, AU-2, SI-4	Reportes de Trivy y traza de la API

Nota. Elaboración propia a partir de Cloud Security Alliance (2024), ISO/IEC (2022b) y Joint Task Force (2020).

Validación técnica

Los componentes del taller se validaron en un entorno Linux independiente antes de su publicación. Las pruebas realizadas fueron cuatro:

1. *Análisis estático.* Los siete scripts de Bash se revisaron con ShellCheck sin advertencias. Las siete políticas en formato JSON y el archivo `docker-compose.yml` se validaron sintácticamente.
2. *Ejecución sobre Moto 5.2.3 con AWS CLI 1.46.* Se ejecutaron los scripts de población y de auditoría del Laboratorio 4 contra un servidor Moto real. El auditor reportó los tres hallazgos esperados y el gestor de secretos devolvió el valor almacenado (Anexo K).
3. *Generación del escenario de secretos.* Se ejecutó el script de preparación con un cliente de MinIO simulado. Se confirmó que el repositorio resultante contiene la credencial en el primer commit y no en el último. Esta prueba detectó un error real: con la opción `pipefail`, la generación aleatoria de claves terminaba el script por la señal SIGPIPE. El error se corrigió antes de la publicación.
4. *Revisión de vigencia.* Se verificaron las condiciones de distribución vigentes en octubre de 2026 de LocalStack, MinIO y Moto, que motivaron la selección de componentes descrita en la metodología.

Como limitación, el entorno de validación no disponía de un servicio Docker ni de acceso a las descargas de MinIO. Por ello, los comandos del cliente `mc` de los Laboratorios 2, 3 y 5 y el escaneo con Trivy deben ejecutarse una vez en el equipo final antes del taller. La guía del facilitador (Anexo J) incluye esta verificación como requisito previo.

Conclusiones

El taller demuestra que es posible construir un laboratorio completo de seguridad en la nube sobre un único equipo con Kali Linux, sin costo y sin exponer sistemas de terceros. La combinación de un almacenamiento de objetos con políticas que se aplican efectivamente (MinIO) y un emulador de las API de AWS (Moto) cubre nueve de las once amenazas principales identificadas por la Cloud Security Alliance en 2024.

El ciclo de provocar, explotar, detectar y corregir convierte cada amenaza en una experiencia observable. El participante no solo conoce el control normalizado: comprueba qué ocurre cuando

falta y verifica que funciona cuando se aplica. La matriz de trazabilidad vincula cada práctica con los controles de ISO/IEC 27002:2022 y NIST SP 800-53, lo que facilita integrar el taller en programas de formación alineados con un sistema de gestión de seguridad de la información.

Los cambios recientes de licenciamiento en LocalStack y MinIO evidencian que la dependencia de componentes de código abierto también es un riesgo de seguridad de la información. El taller incorpora esa lección en su propio diseño: fija las versiones, documenta las alternativas y analiza la imagen desactualizada como parte de la práctica.

Como trabajo futuro se propone ampliar el taller a una segunda sesión con orquestación de contenedores, cifrado gestionado con un servicio de claves y detección automatizada de amenazas.

Referencias

Amazon Web Services. (2026a). *Blocking public access to your Amazon S3 storage*. Consultado el 10 de octubre de 2026, desde https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

Amazon Web Services. (2026b). *Policy evaluation logic*. Consultado el 10 de octubre de 2026, desde https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html

Amazon Web Services. (2026c). *Shared responsibility model*. Consultado el 10 de octubre de 2026, desde https://aws.amazon.com/compliance/shared-responsibility-model/

Amazon Web Services. (2026d). *Use the Instance Metadata Service to access instance metadata*. Consultado el 10 de octubre de 2026, desde https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html

Aqua Security. (2026). *Trivy: Find vulnerabilities, misconfigurations, secrets, SBOM in containers, Kubernetes, code repositories, clouds and more*. Consultado el 10 de octubre de 2026, desde https://github.com/aquasecurity/trivy

Cloud Security Alliance. (2024, 6 de agosto). *Cloud Security Alliance releases Top Threats to Cloud Computing 2024 report*. Consultado el 10 de octubre de 2026, desde https://cloudsecurityalliance.org/press-releases/2024/08/06/cloud-security-alliance-releases-top-threats-to-cloud-computing-2024-report

Congreso de la República de Colombia. (2009, 5 de enero). *Ley 1273 de 2009, por medio de la cual se modifica el Código Penal, se crea un nuevo bien jurídico tutelado denominado "de la protección de la información y de los datos" y se preservan integralmente los sistemas que utilicen las tecnologías de la información y las comunicaciones, entre otras disposiciones*. Consultado el 10 de octubre de 2026, desde http://www.secretariasenado.gov.co/senado/basedoc/ley_1273_2009.html

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Consultado

el 10 de octubre de 2026, desde http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

Docker Inc. (2026). *Packet filtering and firewalls*. Consultado el 10 de octubre de 2026, desde https://docs.docker.com/engine/network/packet-filtering-firewalls/

Gitleaks Contributors. (2026). *Gitleaks: Find secrets with Gitleaks*. Consultado el 10 de octubre de 2026, desde https://github.com/gitleaks/gitleaks

Hu, V. C., Iorga, M., Bao, W., Li, A., Li, Q., & Gouglidis, A. (2020, julio). *General access control guidance for cloud systems* (Special Publication N.º 800-210). National Institute of Standards y Technology. https://doi.org/10.6028/NIST.SP.800-210

International Organization for Standardization & International Electrotechnical Commission. (2015). *ISO/IEC 27017:2015. Information technology: Security techniques: Code of practice for information security controls based on ISO/IEC 27002 for cloud services*. ISO.

International Organization for Standardization & International Electrotechnical Commission. (2022a). *ISO/IEC 27001:2022. Information security, cybersecurity and privacy protection: Information security management systems: Requirements*. ISO.

International Organization for Standardization & International Electrotechnical Commission. (2022b). *ISO/IEC 27002:2022. Information security, cybersecurity and privacy protection: Information security controls*. ISO.

Joint Task Force. (2020, septiembre). *Security and privacy controls for information systems and organizations* (Special Publication N.º 800-53 Rev. 5). National Institute of Standards y Technology. https://doi.org/10.6028/NIST.SP.800-53r5

LocalStack. (2026, 23 de marzo). *Announcing the LocalStack for AWS 2026.03.0 release*. Consultado el 10 de octubre de 2026, desde https://blog.localstack.cloud/localstack-for-aws-release-2026-03-0/

Mell, P., & Grance, T. (2011, septiembre). *The NIST definition of cloud computing* (Special Publication N.º 800-145). National Institute of Standards y Technology. https://doi.org/10.6028/NIST.SP.800-145

MinIO, Inc. (2025). *MinIO: High performance object storage* (Ver. RELEASE.2025-09-07T16-13-09Z). Consultado el 10 de octubre de 2026, desde https://github.com/minio/minio

MITRE Corporation. (2026). *ATT&CK matrix for enterprise: Cloud*. Consultado el 10 de octubre de 2026, desde https://attack.mitre.org/matrices/enterprise/cloud/

Moto Developers. (2026). *Non-Python SDK's / server mode*. Consultado el 10 de octubre de 2026, desde https://docs.getmoto.org/en/latest/docs/server_mode.html

OffSec Services Limited. (2026). *Installing Docker on Kali Linux*. Consultado el 10 de octubre de 2026, desde https://www.kali.org/docs/containers/installing-docker-on-kali/

OWASP Foundation. (2026). *Secrets management cheat sheet*. Consultado el 10 de octubre de 2026, desde https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Souppaya, M., Morello, J., & Scarfone, K. (2017, septiembre). *Application container security guide* (Special Publication N.º 800-190). National Institute of Standards y Technology. https://doi.org/10.6028/NIST.SP.800-190

Anexo A. Estructura del repositorio y publicación en GitHub

El taller se distribuye como un repositorio de GitHub que funciona como guía para los participantes (https://github.com/[usuario]/taller-cloud-seguridad). Cada carpeta de laboratorio contiene un archivo `README.md` con las instrucciones paso a paso que se resumen en el apartado «Desarrollo de las prácticas». Este documento, en norma APA, se incluye en la carpeta `docs/` del repositorio; el resto del taller se mantiene en archivos Markdown.

```
taller-cloud-seguridad/
|-- README.md                    Presentación, agenda e inicio rápido
|-- LICENSE                      MIT (código) y CC BY-SA 4.0 (contenido)
|-- docker-compose.yml           Definición de la nube privada (Anexo C)
|-- .env.example                 Credenciales de ejemplo (Anexo C)
|-- .gitignore                   Exclusión de secretos (Anexo C)
|-- scripts/
|   |-- instalar-herramientas.sh Instalación en Kali Linux (Anexo B)
|   |-- verificar-entorno.sh     Verificación previa (Anexo B)
|   |-- lab.sh                   Control del laboratorio (Anexo D)
|   `-- entorno.sh               Variables y alias (Anexo D)
|-- docs/
|   |-- 00-preparacion.md        Preparación del entorno (Anexo B)
|   |-- guia-facilitador.md      Guía del facilitador (Anexo J)
|   `-- Taller_Nube_Privada_Seguridad_APA.pdf   Este documento
|-- labs/
|   |-- 01-nube-privada/         Laboratorio 1
|   |-- 02-almacenamiento/       Laboratorio 2
|   |-- 03-iam-minio/politicas/  Laboratorio 3 (Anexo E)
|   |-- 04-secretos/             Laboratorio 4 (Anexos E y F)
|   `-- 05-auditoria/terraform/  Laboratorio 5 (Anexo G)
|-- soluciones/README.md         Solucionario (Anexo H)
`-- recursos/checklist-seguridad.md   Lista de verificación (Anexo I)
```

Para publicar el repositorio se crea un repositorio vacío en GitHub y se ejecutan los siguientes comandos desde la carpeta del taller:

```
cd taller-cloud-seguridad
git init
git add .
git commit -m "Taller de nube privada y seguridad cloud"
git branch -M main
git remote add origin https://github.com/[usuario]/taller-cloud-seguridad.git
git push -u origin main
```

Antes del primer `push` se comprueba con `git status` que el archivo `.env` no forma parte del commit. El archivo `.gitignore` lo excluye (Anexo C), en coherencia con el control de secretos que se practica en el Laboratorio 4.

Anexo B. Preparación del entorno en Kali Linux

Este anexo complementa los requisitos de la Tabla 2. La preparación se realiza antes del taller y requiere entre 20 y 30 minutos con conexión a Internet. El día del taller no se necesita conexión.

Instalación automática

```
git clone https://github.com/[usuario]/taller-cloud-seguridad.git
cd taller-cloud-seguridad
chmod +x scripts/*.sh labs/*/*.sh
./scripts/instalar-herramientas.sh
newgrp docker
./scripts/verificar-entorno.sh
```

La pertenencia al grupo `docker` equivale a privilegios de administrador sobre el equipo. Es aceptable para un laboratorio, pero en servidores de producción se prefiere ejecutar Docker sin privilegios (modo *rootless*). Esta es la primera decisión de seguridad que el facilitador discute en el taller.

Prueba de funcionamiento

```
./scripts/lab.sh up
source scripts/entorno.sh
mc admin info nube              # un servidor en línea
awsm sts get-caller-identity    # cuenta 123456789012
./scripts/lab.sh down
```

Problemas frecuentes

Tabla 9
Problemas frecuentes durante la preparación y su solución

Síntoma	Causa probable	Solución
`permission denied ... docker.sock`	No se recargó el grupo `docker`	Ejecutar `newgrp docker` o cerrar la sesión
`port is already allocated`	Otro programa usa el puerto 9000, 9001 o 5000	Identificarlo con `sudo ss -tlnp` y detenerlo
`mc` abre un gestor de archivos	Midnight Commander comparte el nombre `mc`	Usar `/usr/local/bin/mc`; los scripts ya lo priorizan
Paquete `gitleaks` o `trivy` no encontrado	Índices de paquetes desactualizados	`sudo apt update && sudo apt full-upgrade`
MinIO no inicia	Contraseña raíz de menos de 8 caracteres	Ajustar `MINIO_ROOT_PASSWORD` en `.env`

Nota. Elaboración propia.

Scripts de instalación y verificación

Archivo: `scripts/instalar-herramientas.sh`

```
#!/usr/bin/env bash
# Instala todo lo necesario en Kali Linux. Ejecutar UNA vez, antes del taller, con internet.
set -euo pipefail

if [[ $EUID -eq 0 ]]; then
  echo "[x] Este script se ejecuta con el usuario normal (usará sudo cuando haga falta)." >&2
  exit 1
fi

echo "[*] Actualizando índices de paquetes..."
sudo apt-get update

echo "[*] Instalando Docker, AWS CLI y herramientas de seguridad..."
sudo apt-get install -y docker.io docker-compose awscli jq curl git nmap gitleaks trivy

echo "[*] Habilitando Docker y agregando a $USER al grupo docker..."
```

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"

# "mc" también es Midnight Commander: se comprueba que sea el cliente de MinIO.
if ! mc --version 2>/dev/null | grep -qi minio; then
  echo "[*] Instalando el cliente de MinIO (mc)..."
  arq="$(dpkg --print-architecture)"   # amd64 o arm64
  tmp="$(mktemp -d)"
  if curl -fL "https://dl.min.io/client/mc/release/linux-${arq}/mc" -o "$tmp/mc"; then
    sudo install -m 0755 "$tmp/mc" /usr/local/bin/mc
  else
    echo "[!] No se pudo descargar mc. Alternativa: usa 'docker exec nube-minio mc ...'" >&2
  fi
  rm -rf "$tmp"
fi

echo "[*] Descargando imágenes del laboratorio (para trabajar sin internet el día del taller)..."
sudo docker pull minio/minio:RELEASE.2025-09-07T16-13-09Z
sudo docker pull motoserver/moto:latest

echo "[*] Precargando la base de vulnerabilidades de Trivy..."
trivy image --download-db-only || echo "[!] Trivy descargará su base en el Lab 5."

echo
echo "[+] Listo. Se debe cerrar sesión y volver a entrar (o ejecutar 'newgrp docker')"
echo "    para usar Docker sin sudo. Después: ./scripts/verificar-entorno.sh"
```

Archivo: `scripts/verificar-entorno.sh`

```
#!/usr/bin/env bash
# Comprueba que la máquina está lista para el taller. No modifica nada.
set -uo pipefail

ok=0; fallos=0
check() {  # check "descripción" comando...
  local desc="$1"; shift
  if "$@" >/dev/null 2>&1; then
    printf '  [OK]    %s\n' "$desc"; ok=$((ok+1))
  else
    printf '  [FALLA] %s\n' "$desc"; fallos=$((fallos+1))
  fi
}

echo "== Herramientas =="
```

```
for h in docker aws mc jq curl git nmap gitleaks trivy; do
  check "$h instalado" command -v "$h"
done
check "Docker Compose disponible" bash -c 'docker compose version || docker-compose version'
check "Docker usable sin sudo" docker info

echo "== Recursos =="
ram_mb=$(awk '/MemTotal/ {print int($2/1024)}' /proc/meminfo)
check "RAM >= 4 GB (disponible: ${ram_mb} MB)" test "$ram_mb" -ge 4000
disco_gb=$(df -BG --output=avail "$HOME" | tail -1 | tr -dc '0-9')
check "Disco libre >= 5 GB (disponible: ${disco_gb} GB)" test "$disco_gb" -ge 5

echo "== Puertos libres (si el laboratorio NO está corriendo) =="
for p in 9000 9001 5000; do
  if ss -tln | grep -q ":$p "; then
    if docker ps --format '{{.Names}}' 2>/dev/null | grep -q '^nube-'; then
      printf '  [OK]   puerto %s (usado por el laboratorio)\n' "$p"; ok=$((ok+1))
    else
      printf '  [FALLA] puerto %s ocupado por otro programa\n' "$p"; fallos=$((fallos+1))
    fi
  else
    printf '  [OK]   puerto %s libre\n' "$p"; ok=$((ok+1))
  fi
done

echo
echo "Resultado: $ok correctos, $fallos con problemas."
[[ $fallos -eq 0 ]] && echo "La máquina Kali está lista para el taller." || echo "Consultar el Anexo B del documento (docs/00-preparacion.md)"
exit "$fallos"
```

Anexo C. Definición de la nube privada

Este anexo contiene la definición de la arquitectura de la Figura 2. El archivo `docker-compose.yml` limita la publicación de puertos a la interfaz local (Laboratorio 1) y fija la versión de la imagen de MinIO que se escanea en el Laboratorio 5. El archivo `.env.example` contiene valores de ejemplo, y el archivo `.gitignore` impide subir el archivo `.env` real al repositorio.

Archivo: `docker-compose.yml`

```yaml
# Nube privada del taller: almacenamiento S3 (MinIO) + APIs de AWS emuladas (Moto)
# Todos los puertos se publican SOLO en 127.0.0.1: la nube no queda expuesta a la red.

name: nube-taller

services:
  minio:
    # Última imagen comunitaria publicada por MinIO (la comunidad pasó a "solo código fuente"
    # en octubre de 2025). Se fija a propósito: en el Lab 5 se escanea con Trivy.
    image: minio/minio:RELEASE.2025-09-07T16-13-09Z
    container_name: nube-minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER:?Falta MINIO_ROOT_USER en .env}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:?Falta MINIO_ROOT_PASSWORD en .env}
    ports:
      - "127.0.0.1:9000:9000"   # API S3
      - "127.0.0.1:9001:9001"   # Consola web
    volumes:
      - minio-datos:/data
    networks: [nube]
    healthcheck:
      test: ["CMD", "mc", "ready", "local"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

  moto:
    # Emulador de APIs de AWS (IAM, STS, Secrets Manager, S3...). Licencia Apache 2.0, sin token.
    image: motoserver/moto:latest
    container_name: nube-moto
```

```
    environment:
      MOTO_PORT: "5000"
    ports:
      - "127.0.0.1:5000:5000"
    networks: [nube]
    restart: unless-stopped

networks:
  nube:
    driver: bridge

volumes:
  minio-datos:
```

Archivo: `.env.example`

```
# Este archivo se copia como .env y se cambian los valores:  cp .env.example .env
# .env está en .gitignore: nunca debe subirse a GitHub (tema del Lab 4).

# Credenciales "root" de MinIO (equivalen a la cuenta raíz de un proveedor cloud)
MINIO_ROOT_USER=admin-taller
MINIO_ROOT_PASSWORD=CambiaEstaClave-2026!

# Credenciales ficticias para Moto (no son reales, Moto acepta cualquier valor)
AWS_ACCESS_KEY_ID=testing
AWS_SECRET_ACCESS_KEY=testing
AWS_DEFAULT_REGION=us-east-1
```

Archivo: `.gitignore`

```
# Secretos y credenciales
.env
*.pem
*.key
credentials
.aws/

# Artefactos generados durante los laboratorios
labs/04-secretos/app-vulnerable/
labs/05-auditoria/reportes/
labs/**/*.tfstate*
labs/**/.terraform/
*.log
descargas/
```

Anexo D. Scripts de operación del laboratorio

El script `lab.sh` controla el ciclo de vida de la nube privada con las órdenes `up`, `down`, `reset`, `status` y `logs`. La orden `reset` borra todos los datos y deja el laboratorio en su estado inicial. El script `entorno.sh` se carga con `source` en cada terminal. Define las variables de entorno, el alias `awsm` para Moto y el alias `nube` del cliente de MinIO, que se utilizan en todos los laboratorios de el apartado «Desarrollo de las prácticas».

Archivo: `scripts/lab.sh`

```
#!/usr/bin/env bash
# Control del laboratorio: ./scripts/lab.sh {up|down|reset|status|logs}
set -euo pipefail

RAIZ="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$RAIZ"

# Kali puede traer "docker compose" (plugin) o "docker-compose" (binario). Se usa el
    que exista.
if docker compose version >/dev/null 2>&1; then
  COMPOSE=(docker compose)
elif command -v docker-compose >/dev/null 2>&1; then
  COMPOSE=(docker-compose)
else
  echo "[x] No se encontró Docker Compose. Se debe ejecutar: ./scripts/instalar-
    herramientas.sh" >&2
  exit 1
fi

if [[ ! -f .env ]]; then
  echo "[!] No existe .env; se crea desde .env.example. La contraseña debe cambiarse
    antes del taller."
  cp .env.example .env
fi

esperar_minio() {
  echo -n "[*] Esperando a MinIO"
  for _ in $(seq 1 30); do
    if curl -fsS http://localhost:9000/minio/health/live >/dev/null 2>&1; then
      echo " listo."
      return 0
    fi
    echo -n "."
    sleep 2
```

```
  done
  echo
  echo "[x] MinIO no respondió en 60 s. Revisar: ./scripts/lab.sh logs" >&2
  return 1
}

case "${1:-}" in
  up)
    "${COMPOSE[@]}" up -d
    esperar_minio
    echo "[+] Nube privada en marcha:"
    echo "    API S3 (MinIO) : http://localhost:9000"
    echo "    Consola MinIO  : http://localhost:9001"
    echo "    APIs AWS (Moto): http://localhost:5000  (panel: http://localhost:5000/
    moto-api/)"
    echo "    Siguiente paso : source scripts/entorno.sh"
    ;;
  down)
    "${COMPOSE[@]}" down
    ;;
  reset)
    read -r -p "Esto borra TODOS los datos del laboratorio. ?`Continuar? [s/N] " r
    [[ "$r" =~ ^[sS]$ ]] || { echo "Cancelado."; exit 0; }
    "${COMPOSE[@]}" down -v
    rm -rf labs/04-secretos/app-vulnerable labs/05-auditoria/reportes
    "${COMPOSE[@]}" up -d
    esperar_minio
    echo "[+] Laboratorio reiniciado desde cero."
    ;;
  status)
    "${COMPOSE[@]}" ps
    ;;
  logs)
    "${COMPOSE[@]}" logs --tail=50 -f
    ;;
  *)
    echo "Uso: $0 {up|down|reset|status|logs}"
    exit 1
    ;;
esac
```

Archivo: `scripts/entorno.sh`

```
# shellcheck shell=bash
# Cargar en cada terminal nueva:   source scripts/entorno.sh
# (no se ejecuta, se "sourcea": define variables y alias en la shell actual)
```

```
4
5  _RAIZ_TALLER="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
6
7  if [[ -f "$_RAIZ_TALLER/.env" ]]; then
8    set -a
9    # shellcheck disable=SC1091
10   source "$_RAIZ_TALLER/.env"
11   set +a
12 else
13   echo "[x] Falta .env. Se debe ejecutar primero: ./scripts/lab.sh up" >&2
14   return 1 2>/dev/null || exit 1
15 fi
16
17 # AWS CLI apuntando a Moto (la "AWS" local del laboratorio). Nunca toca una cuenta
       real.
18 export AWS_PAGER=""
19 alias awsm='aws --endpoint-url http://localhost:5000'
20
21 # "mc" también es el nombre de Midnight Commander: se prefiere el binario de MinIO.
22 if [[ -x /usr/local/bin/mc ]]; then
23   alias mc=/usr/local/bin/mc
24   _MC=/usr/local/bin/mc
25 else
26   _MC=mc
27 fi
28
29 # Alias "nube" del cliente MinIO con la cuenta root (solo para administrar)
30 if "$_MC" --version 2>/dev/null | grep -qi minio; then
31   "$_MC" alias set nube http://localhost:9000 "$MINIO_ROOT_USER" "$MINIO_ROOT_PASSWORD
       " >/dev/null \
32     && echo "[+] Alias 'nube' de mc configurado (cuenta root)."
33 else
34   echo "[!] El cliente 'mc' de MinIO no está instalado. Se debe ejecutar: ./scripts/
       instalar-herramientas.sh" >&2
35 fi
36 unset _MC
37
38 echo "[+] Entorno cargado. Usa 'awsm' para AWS emulado y 'mc ... nube/' para MinIO."
```

Anexo E. Políticas de control de acceso

Las tres primeras políticas se aplican en el Laboratorio 3 (el apartado «Laboratorio 3: identidad y mínimo privilegio»). La política `demasiado-permisiva` representa el atajo inseguro. La política `analista-lectura-informes` aplica el mínimo privilegio. La política `backup-sin-borrado` demuestra la precedencia del rechazo explícito. Las cuatro restantes se usan en el Laboratorio 4 (el apartado «Laboratorio 4: secretos filtrados y auditoría de identidades»): la cuenta de servicio de la aplicación en MinIO y las políticas de la cuenta de AWS emulada que examina el auditor del Anexo F.

Archivo: `labs/03-iam-minio/politicas/demasiado-permisiva.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LaPoliticaQueNadieDeberiaEscribir",
      "Effect": "Allow",
      "Action": ["s3:*"],
      "Resource": ["arn:aws:s3:::*"]
    }
  ]
}
```

Archivo: `labs/03-iam-minio/politicas/analista-lectura-informes.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListarSoloElBucketDeInformes",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": ["arn:aws:s3:::informes"]
    },
    {
      "Sid": "LeerObjetosDeInformes",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": ["arn:aws:s3:::informes/*"]
    }
```

```
  ]
}
```

Archivo: `labs/03-iam-minio/politicas/backup-sin-borrado.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EscribirYLeerBackups",
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject", "s3:PutObject"],
      "Resource": ["arn:aws:s3:::backups", "arn:aws:s3:::backups/*"]
    },
    {
      "Sid": "NuncaBorrarBackups",
      "Effect": "Deny",
      "Action": ["s3:DeleteObject", "s3:DeleteObjectVersion", "s3:DeleteBucket"],
      "Resource": ["arn:aws:s3:::backups", "arn:aws:s3:::backups/*"]
    }
  ]
}
```

Archivo: `labs/04-secretos/politicas/app-web.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject", "s3:PutObject"],
      "Resource": ["arn:aws:s3:::app-clientes", "arn:aws:s3:::app-clientes/*"]
    }
  ]
}
```

Archivo: `labs/04-secretos/politicas/admin-todo.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "*", "Resource": "*" }
  ]
}
```

Archivo: `labs/04-secretos/politicas/dev-gestiona-iam.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LeerTodo",
      "Effect": "Allow",
      "Action": ["s3:Get*", "s3:List*", "ec2:Describe*"],
      "Resource": "*"
    },
    {
      "Sid": "ParaQueSeAutoGestioneSusLlaves",
      "Effect": "Allow",
      "Action": ["iam:CreateAccessKey", "iam:AttachUserPolicy", "iam:PutUserPolicy"],
      "Resource": "*"
    }
  ]
}
```

Archivo: `labs/04-secretos/politicas/lectura-facturacion.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::facturacion", "arn:aws:s3:::facturacion/*"]
    }
  ]
}
```

Anexo F. Scripts del escenario de secretos y auditoría de identidades

Estos scripts sustentan el Laboratorio 4 (el apartado «Laboratorio 4: secretos filtrados y auditoría de identidades»). El script `preparar-escenario.sh` genera claves aleatorias con formato de AWS, para que los escáneres de secretos las reconozcan, y construye el repositorio con la filtración en su historial. El script `poblar-iam-aws.sh` crea en Moto los usuarios, las políticas del Anexo E y un secreto almacenado correctamente. El script `auditar-iam.sh` revisa las políticas en modo de solo lectura. Su lógica también es aplicable a una cuenta real si se elimina el parámetro `--endpoint-url`. Su salida real se presenta en el Anexo K.

Archivo: `labs/04-secretos/preparar-escenario.sh`

```
#!/usr/bin/env bash
# Crea el escenario del Lab 4:
#  1) Un usuario de servicio en MinIO ("la app web") con acceso a un bucket con datos de clientes.
#  2) Un repositorio git "de la app" donde un desarrollador subió esas credenciales
#     y luego las "borró" en otro commit (siguen en el historial).
# Requiere: laboratorio en marcha y 'source scripts/entorno.sh' cargado.
set -euo pipefail

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO="$DIR/app-vulnerable"

# Se usa el cliente de MinIO aunque Midnight Commander también se llame "mc".
mc() { if [[ -x /usr/local/bin/mc ]]; then /usr/local/bin/mc "$@"; else command mc "$@"; fi; }

mc alias list nube >/dev/null 2>&1 || { echo "[x] Primero se debe cargar: source scripts/entorno.sh" >&2; exit 1; }
[[ -d "$REPO" ]] && { echo "[!] El escenario ya existe en $REPO (se borra la carpeta para regenerarlo)."; exit 0; }

# Clave con formato de AWS (AKIA + 16 caracteres) para que los escáneres la reconozcan.
# (se lee un bloque fijo de /dev/urandom para no romper 'set -o pipefail' con SIGPIPE)
aleatorio() { head -c 4096 /dev/urandom | LC_ALL=C tr -dc "$1" | cut -c "1-$2"; }
ACCESS_KEY="AKIA$(aleatorio 'A-Z2-7' 16)"
SECRET_KEY="$(aleatorio 'A-Za-z0-9' 40)"

echo "[*] Creando bucket con datos ficticios de clientes..."
mc mb --ignore-existing nube/app-clientes >/dev/null
```

```
tmp="$(mktemp -d)"
cat >"$tmp/clientes.csv" <<'CSV'
id,nombre,correo,ciudad
1,Ana Pérez,ana.perez@ejemplo.test,Bogotá
2,Luis Gómez,luis.gomez@ejemplo.test,Medellín
3,Marta Ruiz,marta.ruiz@ejemplo.test,Cali
CSV
mc cp "$tmp/clientes.csv" nube/app-clientes/ >/dev/null
rm -rf "$tmp"

echo "[*] Creando la cuenta de servicio de la app..."
mc admin policy create nube app-web "$DIR/politicas/app-web.json" >/dev/null
mc admin user add nube "$ACCESS_KEY" "$SECRET_KEY" >/dev/null
mc admin policy attach nube app-web --user "$ACCESS_KEY" >/dev/null

echo "[*] Creando el repositorio de la app con el error del desarrollador..."
mkdir -p "$REPO"
cd "$REPO"
git init -q
git config user.name "Dev Apurado"
git config user.email "dev@ejemplo.test"

cat >app.py <<'PY'
import boto3
from config import S3_ENDPOINT, ACCESS_KEY, SECRET_KEY

s3 = boto3.client("s3", endpoint_url=S3_ENDPOINT,
                  aws_access_key_id=ACCESS_KEY, aws_secret_access_key=SECRET_KEY)
print(s3.list_objects_v2(Bucket="app-clientes"))
PY
cat >config.py <<PY
# TODO: mover a variables de entorno antes de producción
S3_ENDPOINT = "http://localhost:9000"
ACCESS_KEY = "$ACCESS_KEY"
SECRET_KEY = "$SECRET_KEY"
PY
git add . && git commit -qm "Primera versión de la app"

cat >config.py <<'PY'
import os
S3_ENDPOINT = os.environ["S3_ENDPOINT"]
ACCESS_KEY = os.environ["ACCESS_KEY"]
SECRET_KEY = os.environ["SECRET_KEY"]
PY
git add . && git commit -qm "Quitar credenciales del código (ya está seguro)"

echo "[+] Escenario listo en: $REPO"
```

```
echo "    El último commit ya no tiene credenciales... ?`o sí siguen en algún lado?"
```

Archivo: `labs/04-secretos/poblar-iam-aws.sh`

```
#!/usr/bin/env bash
# Puebla la "cuenta AWS" emulada (Moto) con usuarios y políticas, algunas peligrosas.
# Así el Lab 4 tiene algo realista que auditar. Requiere el laboratorio en marcha.
set -euo pipefail

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
export AWS_ACCESS_KEY_ID=testing AWS_SECRET_ACCESS_KEY=testing AWS_DEFAULT_REGION=us-east-1 AWS_PAGER=""
AWS=(aws --endpoint-url http://localhost:5000)
CUENTA="123456789012"   # cuenta por defecto de Moto

crear_politica() {  # nombre archivo
  "${AWS[@]}" iam create-policy --policy-name "$1" \
    --policy-document "file://$DIR/politicas/$2" >/dev/null 2>&1 || true
}

echo "[*] Creando políticas..."
crear_politica AdminTodo          admin-todo.json
crear_politica DevGestionaIAM     dev-gestiona-iam.json
crear_politica LecturaFacturacion lectura-facturacion.json

echo "[*] Creando usuarios y asignando políticas..."
for u in carla-admin diego-dev elena-finanzas bot-ci; do
  "${AWS[@]}" iam create-user --user-name "$u" >/dev/null 2>&1 || true
done
"${AWS[@]}" iam attach-user-policy --user-name carla-admin    --policy-arn "arn:aws:iam::$CUENTA:policy/AdminTodo"
"${AWS[@]}" iam attach-user-policy --user-name diego-dev      --policy-arn "arn:aws:iam::$CUENTA:policy/DevGestionaIAM"
"${AWS[@]}" iam attach-user-policy --user-name elena-finanzas --policy-arn "arn:aws:iam::$CUENTA:policy/LecturaFacturacion"
"${AWS[@]}" iam attach-user-policy --user-name bot-ci         --policy-arn "arn:aws:iam::$CUENTA:policy/AdminTodo"
"${AWS[@]}" iam create-access-key --user-name bot-ci >/dev/null

echo "[*] Guardando un secreto en Secrets Manager (la forma correcta)..."
"${AWS[@]}" secretsmanager create-secret --name prod/app/base-datos \
  --secret-string '{"usuario":"app","clave":"N0-En-El-Codigo!"}' >/dev/null 2>&1 || true

echo "[+] Cuenta AWS emulada poblada. Explorar con: awsm iam list-users"
```

Archivo: `labs/04-secretos/auditar-iam.sh`

```
#!/usr/bin/env bash
# Auditor mínimo de IAM: revisa las políticas gestionadas por el cliente y marca
#  - comodines totales (Action "*" sobre Resource "*")
#  - acciones IAM que permiten escalar privilegios
#  - usuarios con llaves de acceso y política de administrador
# Funciona contra Moto; con una cuenta real bastaría quitar --endpoint-url (solo lectura).
set -euo pipefail

export AWS_PAGER=""
ENDPOINT="${ENDPOINT:-http://localhost:5000}"
AWS=(aws --endpoint-url "$ENDPOINT")
PELIGROSAS='["iam:*","iam:CreateAccessKey","iam:AttachUserPolicy","iam:PutUserPolicy","iam:CreatePolicyVersion","iam:PassRole","iam:UpdateAssumeRolePolicy"]'

hallazgos=0
echo "== Políticas gestionadas por el cliente =="
for arn in $("${AWS[@]}" iam list-policies --scope Local --query 'Policies[].Arn' --output text); do
  ver=$("${AWS[@]}" iam get-policy --policy-arn "$arn" --query 'Policy.DefaultVersionId' --output text)
  doc=$("${AWS[@]}" iam get-policy-version --policy-arn "$arn" --version-id "$ver" \
        --query 'PolicyVersion.Document' --output json)
  # Algunas versiones devuelven el documento como texto (URL-encoded); se normaliza.
  if [[ "$(jq -r 'type' <<<"$doc")" == "string" ]]; then
    doc=$(jq -r '.' <<<"$doc" | python3 -c 'import sys,urllib.parse;print(urllib.parse.unquote(sys.stdin.read()))')
  fi
  # Normaliza Statement/Action/Resource a listas para poder compararlos.
  norm=$(jq '[.Statement] | flatten | map(select(.Effect=="Allow")
              | {a: ([.Action] | flatten), r: ([.Resource] | flatten)})' <<<"$doc")

  nombre="${arn##*/}"
  if jq -e 'any(.[]; (.a | index("*")) and (.r | index("*")))' <<<"$norm" >/dev/null; then
    echo "  [CRÍTICO] $nombre concede *:* (administrador total)"; hallazgos=$((hallazgos+1))
  fi
  escalada=$(jq -r --argjson p "$PELIGROSAS" '[.[].a[] | select(. as $x | $p | index($x))] | unique | join(", ")' <<<"$norm")
  if [[ -n "$escalada" ]]; then
    echo "  [ALTO]    $nombre permite escalar privilegios: $escalada"; hallazgos=$((hallazgos+1))
  fi
done
```

```
37
38 echo "== Usuarios =="
39 for u in $("${AWS[@]}" iam list-users --query 'Users[].UserName' --output text); do
40   pols=$("${AWS[@]}" iam list-attached-user-policies --user-name "$u" --query '
       AttachedPolicies[].PolicyName' --output text)
41   llaves=$("${AWS[@]}" iam list-access-keys --user-name "$u" --query 'length(
       AccessKeyMetadata)' --output text)
42   echo "  $u -> políticas: ${pols:-ninguna} | llaves de acceso: $llaves"
43   if [[ "$pols" == *AdminTodo* && "$llaves" -gt 0 ]]; then
44     echo "  [ALTO]    $u tiene llaves de larga duración Y es administrador"; hallazgos
       =$((hallazgos+1))
45   fi
46 done
47
48 echo
49 echo "Total de hallazgos: $hallazgos"
```

Anexo G. Infraestructura como código con errores deliberados

La plantilla se analiza en el Laboratorio 5 (el apartado «Laboratorio 5: auditoría y aseguramiento») y no se despliega en ningún proveedor. Contiene siete errores de seguridad comentados en el propio código: bloqueo de acceso público desactivado, acceso SSH abierto a Internet, salida de red sin restricción, servicio de metadatos en su versión 1, disco raíz sin cifrar, contraseña en texto plano en los datos de usuario y política IAM con comodín total. La corrección esperada de cada uno se presenta en la Tabla 10 del Anexo H.

Archivo: `labs/05-auditoria/terraform/main.tf`

```
# Infraestructura como código CON ERRORES A PROPÓSITO para el Lab 5.
# No se despliega: solo se analiza con "trivy config". El participante identifica y
    corrige los problemas.

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# --- Almacenamiento ------------------------------------------------------
resource "aws_s3_bucket" "fotos_clientes" {
  bucket = "taller-fotos-clientes"
}

# Error: el bloqueo de acceso público está desactivado
resource "aws_s3_bucket_public_access_block" "fotos_clientes" {
  bucket                  = aws_s3_bucket.fotos_clientes.id
  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}

# Error: sin cifrado, sin versionado y sin registro de accesos (no hay recursos para
```

```
    ello)

# --- Red ---------------------------------------------------------------
resource "aws_security_group" "servidor_web" {
  name        = "servidor-web"
  description = "Servidor web del taller"

  # Error: SSH abierto a todo Internet
  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Error: salida sin ninguna restricción
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# --- Cómputo -----------------------------------------------------------
resource "aws_instance" "web" {
  ami                    = "ami-0123456789abcdef0"
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.servidor_web.id]

  # Error: IMDSv1 permitido (robo de credenciales vía SSRF al servicio de metadatos)
  metadata_options {
    http_tokens = "optional"
  }

  # Error: disco raíz sin cifrar
  root_block_device {
    encrypted = false
```

```
  }

  # Error: secreto en texto plano dentro de user_data
  user_data = <<-EOT
    #!/bin/bash
    export DB_PASSWORD="SuperSecreta123"
    systemctl start app
  EOT
}

# --- Identidad ---------------------------------------------------------
resource "aws_iam_policy" "app" {
  name = "app-politica"

  # Error: comodín total
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = "*"
      Resource = "*"
    }]
  })
}
```

Anexo H. Solucionario de las preguntas de análisis

Las respuestas pueden variar ligeramente según las versiones de las herramientas. El facilitador entrega este anexo a los participantes al finalizar el taller.

Laboratorio 1

1. Cada contenedor recibe una dirección en la red `nube-taller_nube`, normalmente en el rango `172.18.0.0/16`. Los contenedores se comunican por nombre de servicio (`minio`, `moto`) gracias al servicio de nombres interno de Docker.
2. La cabecera `Server: MinIO` revela el producto. Con la versión, un atacante puede buscar vulnerabilidades conocidas y ajustar su ataque.
3. Grupos de seguridad que limiten el origen, subredes privadas sin dirección pública, listas de control de acceso de red y el acceso administrativo mediante una red privada virtual o un bastión, en lugar de publicar la consola.

Laboratorio 2

1. `download` permite solo leer objetos; `upload`, solo escribir; `public`, leer, escribir y listar. Un contenedor con escritura anónima permite alterar contenidos, alojar software malicioso o generar costos de almacenamiento.
2. La traza muestra la dirección de origen, la operación de la API, el contenedor, el objeto, el código de respuesta y la ausencia de credencial.
3. La firma de la URL es un código de autenticación calculado con la clave secreta sobre todos los parámetros, incluidos la fecha y la duración. Cualquier alteración invalida la firma.

Laboratorio 3

1. El control total y permanente de la nube: crear usuarios, modificar políticas y borrar todos los datos, sin que sea posible restringirlo con políticas. La buena práctica consiste en no crear claves de acceso para la cuenta raíz, protegerla con autenticación multifactor y reservarla para

las tareas que la exigen.

2. `s3:ListBucket` es una operación sobre el contenedor y `s3:GetObject` es una operación sobre los objetos que contiene. Si se omite el recurso correspondiente, falla el listado o falla la lectura.

3. Toda solicitud está denegada por defecto. Un permiso la habilita, pero un rechazo explícito anula cualquier permiso, sin importar la política de origen (Amazon Web Services, 2026b). Por eso el robot no puede borrar copias. Sin embargo, mientras conserve la política `readwrite` sí puede leer el contenedor `rrhh`, porque el rechazo solo cubre el borrado en `backups`.

Laboratorio 4

1. Git conserva el historial completo. El segundo commit solo modifica la versión vigente del archivo. Si el repositorio fuera público, la credencial debería considerarse comprometida desde el primer `push`.

2. La acción `secretsmanager:GetSecretValue`, sobre el identificador exacto del secreto `prod/app/base-datos`. Frente a un archivo `.env`, el gestor ofrece control de acceso por identidad, auditoría de cada lectura, cifrado y rotación automática (OWASP Foundation, 2026).

3. En dos pasos. Primero, con `iam:AttachUserPolicy`, se asigna a sí mismo la política administrada `AdministratorAccess`. Segundo, con `iam:CreateAccessKey`, crea una clave nueva para conservar el acceso. La corrección consiste en retirar las acciones de IAM de su política o limitar `iam:CreateAccessKey` a su propio usuario mediante la variable `${aws:username}`.

Laboratorio 5

1. Compilar el componente desde el código fuente y mantener una imagen propia, adoptar una bifurcación mantenida por la comunidad, migrar a otra solución compatible con S3 o contratar la edición comercial. Mientras tanto, las vulnerabilidades sin corregir se acumulan en la imagen y en sus dependencias.

2. Son útiles la identidad, la operación, el recurso, el código de respuesta 403 y la dirección de origen. En AWS, esta función la cumplen AWS CloudTrail, con eventos de datos de S3, junto con Amazon GuardDuty o las reglas del sistema de gestión de eventos de seguridad.

Tabla 10
Hallazgos y correcciones esperadas en la infraestructura como código

Recurso	**Problema**	**Corrección**
Bloqueo de acceso público	Los cuatro parámetros en `false`	Los cuatro parámetros en `true` (Amazon Web Services, 2026a)
Contenedor S3	Sin cifrado, versionado ni registro de accesos	Añadir los recursos de cifrado, versionado y registro
Grupo de seguridad	SSH desde `0.0.0.0/0` y salida sin restricción	SSH solo desde la red administrativa; restringir la salida
Instancia: metadatos	`http_tokens = "optional"` (versión 1)	`http_tokens = "required"` (Amazon Web Services, 2026d)
Instancia: disco	`encrypted = false`	`encrypted = true`
Instancia: datos de usuario	Contraseña en texto plano	Leerla del gestor de secretos al iniciar
Política IAM	Acción y recurso *	Acciones y recursos específicos

Nota. Elaboración propia.

Anexo I. Lista de verificación de seguridad en la nube

La lista se utiliza en el cierre del taller como instrumento de autoevaluación (el apartado «Evaluación del aprendizaje»). Cada participante marca los controles que su organización ya cumple. Cada bloque corresponde a un laboratorio.

Red y exposición (Laboratorio 1)

- ☐ Los servicios administrativos no se publican en Internet; se accede a ellos mediante una red privada virtual o un bastión.
- ☐ Las reglas de cortafuegos no permiten `0.0.0.0/0` salvo en puertos públicos justificados.
- ☐ Existe un inventario periódico de puertos y direcciones públicas.

Almacenamiento (Laboratorio 2)

- ☐ El bloqueo de acceso público está activado a nivel de cuenta.
- ☐ Los archivos se comparten con URL prefirmadas con caducidad.
- ☐ Los contenedores críticos tienen versionado o bloqueo de objetos.
- ☐ El cifrado en reposo está activado por defecto.

Identidad (Laboratorio 3)

- ☐ La cuenta raíz tiene autenticación multifactor, no tiene claves de acceso y no se usa en la operación diaria.
- ☐ Cada persona y cada aplicación tiene su propia identidad.
- ☐ Las políticas usan acciones y recursos específicos, y cada comodín está justificado.
- ☐ Existen rechazos explícitos para las acciones destructivas.
- ☐ Existe un procedimiento probado para revocar credenciales en minutos.

Secretos (Laboratorio 4)

- ☐ No hay secretos en el código, en las imágenes ni en archivos versionados.
- ☐ Se escanean secretos antes de cada commit y en el repositorio.
- ☐ Los secretos se almacenan en un gestor con rotación.
- ☐ Los procesos automatizados usan credenciales temporales en lugar de claves permanentes.
- ☐ Se auditan los permisos que permiten escalar privilegios.

Auditoría y aseguramiento (Laboratorio 5)

- ☐ Las imágenes se escanean en el proceso de integración y se bloquean las vulnerabilidades críticas.
- ☐ La infraestructura como código se analiza antes de cada despliegue.
- ☐ La versión 2 del servicio de metadatos es obligatoria en todas las instancias.
- ☐ El registro de la API está activado, centralizado y protegido contra borrado.
- ☐ Existen alertas para accesos denegados repetidos y cambios en las identidades.

Anexo J. Guía del facilitador y reto final

Preparación

Una semana antes del taller, el facilitador envía el enlace del repositorio y solicita a los participantes ejecutar los scripts del Anexo B en sus equipos. También prepara una máquina virtual de Kali Linux de respaldo para quienes no cuenten con un entorno funcional. Por último, recorre los cinco laboratorios en un equipo limpio con las versiones vigentes. Este recorrido cubre la verificación pendiente descrita en el apartado «Validación técnica».

Guion por bloques

Tabla 11
Mensajes clave y errores frecuentes por bloque

Bloque	Tiempo	Mensaje clave	Errores frecuentes
Introducción	20 min	Las brechas en la nube se originan sobre todo en configuraciones del cliente	—
Laboratorio 1	30 min	Lo que se publica es superficie de ataque	Grupo `docker` sin recargar; puertos ocupados
Laboratorio 2	45 min	El acceso público casi nunca es la necesidad real	Conflicto con Midnight Commander
Laboratorio 3	40 min	La identidad es el nuevo perímetro; el rechazo explícito prevalece	Contraseñas cortas; omitir `--user`
Laboratorio 4	35 min	Un secreto que llegó a Git está comprometido	Subcomando de gitleaks según su versión
Laboratorio 5	30 min	Se escanea antes de desplegar	Base de Trivy sin descargar
Cierre	15 min	Lista de verificación y reto final	—

Nota. Elaboración propia.

Si el tiempo es insuficiente, pueden omitirse el versionado del Laboratorio 2, la traza del Laboratorio 3 y la parte de registros del Laboratorio 5 sin afectar los objetivos específicos.

Reto final

El facilitador ejecuta el siguiente bloque en el equipo de cada participante, sin mostrarlo. El bloque introduce tres configuraciones inseguras. Se considera superado cuando el participante identifica y corrige las tres con las herramientas `mc anonymous list`, `mc admin user list` y `mc admin user info`. La pista que se entrega es: «hay un contenedor que responde a desconocidos, alguien con demasiado poder y alguien que ya no debería estar».

```
source scripts/entorno.sh
mc mb --ignore-existing nube/reto-auditoria
echo "bandera{bucket_abierto}" | mc pipe nube/reto-auditoria/flag.txt
mc anonymous set download nube/reto-auditoria                # error 1
mc admin user add nube becario 'Becario-2026'
mc admin policy attach nube consoleAdmin --user becario      # error 2
mc admin user add nube ex-empleado 'ExEmpleado-2026'
mc admin policy attach nube readwrite --user ex-empleado     # error 3
```

Al finalizar, los participantes ejecutan `./scripts/lab.sh down`, o `./scripts/lab.sh reset` si desean repetir el taller, y reciben el solucionario del Anexo H.

Anexo K. Evidencias de la validación técnica

Este anexo documenta la salida real de las pruebas descritas en el apartado «Validación técnica», obtenidas el 10 de octubre de 2026 con Moto 5.2.3 y AWS CLI 1.46.1. La primera evidencia corresponde a la ejecución de los scripts del Laboratorio 4 (Anexo F) contra un servidor Moto. Los tres hallazgos coinciden con la evidencia esperada de el apartado «Laboratorio 4: secretos filtrados y auditoría de identidades».

```
$ ./poblar-iam-aws.sh
[*] Creando políticas...
[*] Creando usuarios y asignando políticas...
[*] Guardando un secreto en Secrets Manager (la forma correcta)...
[+] Cuenta AWS emulada poblada. Explorar con: awsm iam list-users

$ ./auditar-iam.sh
== Políticas gestionadas por el cliente ==
  [CRÍTICO] AdminTodo concede *:* (administrador total)
  [ALTO]    DevGestionaIAM permite escalar privilegios: iam:AttachUserPolicy, iam:
    CreateAccessKey, iam:PutUserPolicy
== Usuarios ==
  carla-admin -> políticas: AdminTodo | llaves de acceso: 0
  diego-dev -> políticas: DevGestionaIAM | llaves de acceso: 0
  elena-finanzas -> políticas: LecturaFacturacion | llaves de acceso: 0
  bot-ci -> políticas: AdminTodo | llaves de acceso: 1
  [ALTO]    bot-ci tiene llaves de larga duración Y es administrador

Total de hallazgos: 3

$ awsm secretsmanager get-secret-value --secret-id prod/app/base-datos --query
    SecretString --output text
{"usuario":"app","clave":"N0-En-El-Codigo!"}
```

La segunda evidencia corresponde al análisis estático de los scripts y de las políticas de los Anexo D, Anexo E y Anexo F.

```
$ shellcheck -S warning scripts/*.sh labs/*/*.sh && echo "Sin advertencias"
Sin advertencias

$ for f in $(find . -name "*.json"); do python3 -m json.tool "$f" >/dev/null && echo "
    JSON valido: $f"; done
JSON valido: ./labs/03-iam-minio/politicas/analista-lectura-informes.json
```

```
JSON valido: ./labs/03-iam-minio/politicas/backup-sin-borrado.json
JSON valido: ./labs/03-iam-minio/politicas/demasiado-permisiva.json
JSON valido: ./labs/04-secretos/politicas/admin-todo.json
JSON valido: ./labs/04-secretos/politicas/app-web.json
JSON valido: ./labs/04-secretos/politicas/dev-gestiona-iam.json
JSON valido: ./labs/04-secretos/politicas/lectura-facturacion.json
```